

Automatisation d'une infrastructure cloud avec Terraform

Déploiement d'un serveur AWS EC2 par Infrastructure as Code

Nom : Echikr Samir

Classe : BTS SIO option SISR

1. Introduction

L'objectif de ce projet est de créer et de configurer une infrastructure cloud complète non pas manuellement depuis une console d'administration, mais à partir d'un fichier de configuration. Cette approche, appelée Infrastructure as Code, consiste à décrire l'état souhaité d'une infrastructure dans du code versionnable.

Elle présente trois avantages majeurs : la reproductibilité, puisque le même code produit toujours la même infrastructure ; la traçabilité, chaque modification étant historisée dans un dépôt Git ; et la maîtrise des coûts, l'ensemble des ressources pouvant être détruit par une seule commande.

L'outil utilisé est Terraform, développé par HashiCorp, associé au fournisseur cloud Amazon Web Services.

2. Présentation du code d'infrastructure

Le fichier main.tf décrit l'ensemble des ressources à créer. Il déclare d'abord le fournisseur AWS et la région d'hébergement (Paris, eu-west-3), puis trois ressources : une paire de clés SSH permettant l'accès au serveur, un groupe de sécurité faisant office de pare-feu, et l'instance EC2 elle-même.

Le groupe de sécurité autorise en entrée le port 22 pour l'administration SSH et le port 8000 pour l'application. Une section user_data contient un script exécuté au premier démarrage du serveur : il installe Docker, récupère le code de l'application depuis GitHub et lance le conteneur. Enfin, un bloc output affiche l'adresse IP publique une fois l'infrastructure créée.

```
root@sams-virtual-machine:/home/sams/mon-terraform# cat main.tf
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "eu-west-3"
}

resource "aws_key_pair" "ma_cle" {
  key_name     = "ma-cle-devops"
  public_key   = file("~/ssh/ma-cle-devops.pub")
}

resource "aws_security_group" "mon_firewall" {
  name = "mon-firewall"

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port = 8000
    to_port   = 8000
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

Figure 1 — Extrait du fichier main.tf (provider, clé SSH et règles de pare-feu)

3. Installation des outils

Deux outils sont nécessaires : Terraform, qui interprète le code d'infrastructure, et l'AWS CLI, qui gère l'authentification auprès d'Amazon Web Services. Terraform a été installé depuis le dépôt officiel HashiCorp ajouté aux sources de paquets du système.

```
terraform version
```

```
root@sams-virtual-machine:/home/sams# terraform version
Terraform v1.15.8
on linux_amd64
```

Figure 2 — Version de Terraform installée

```
aws --version
```

```
root@sams-virtual-machine:/home/sams# aws --version
aws-cli/2.36.6 Python/3.14.6 Linux/6.8.0-124-generic exe/x86_64.ubuntu.22
```

Figure 3 — Version de l'AWS CLI installée

4. Configuration de l'accès à AWS

Terraform s'exécutant depuis un poste local, il doit prouver à AWS qu'il est autorisé à créer des ressources. Cette authentification repose sur une paire de clés d'accès (identifiant et clé secrète) générée depuis la console AWS, puis enregistrée localement à l'aide de la commande `aws configure`.

La commande suivante permet de vérifier que l'authentification fonctionne : elle interroge AWS pour connaître l'identité associée aux clés configurées.

```
aws sts get-caller-identity
```

```
root@sams-virtual-machine:/home/sams# aws sts get-caller-identity
{
  "UserId": "AIDAJL4SXXXXXX",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user:sams"
}
```

Figure 4 — Authentification auprès d'AWS vérifiée (identifiant de compte masqué)

5. Récupération du projet

Le code d'infrastructure est hébergé sur un dépôt GitHub et récupéré localement à l'aide de la commande `git clone`.

```
git clone https://github.com/samx77-ux/mon-terraform.git
```

```
root@sams-virtual-machine:/home/sams# git clone https://github.com/samx77-ux/mon-terraform.git
Clonage dans 'mon-terraform'...
remote: Enumerating objects: 13, done.
remote: Counting objects: 100% (13/13), done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 13 (delta 3), reused 9 (delta 2), pack-reused 0 (from 0)
Réception d'objets: 100% (13/13), 6.58 Kio | 842.00 Kio/s, fait.
Résolution des deltas: 100% (3/3), fait.
```

Figure 5 — Clonage du dépôt contenant le code Terraform

Deux opérations de préparation ont ensuite été nécessaires. D'une part, les fichiers d'état présents dans le dépôt ont été supprimés, car ils décrivaient une infrastructure qui n'existait plus. D'autre part, la paire de clés SSH référencée par le code a dû être régénérée, celle d'origine ayant disparu avec l'ancien poste de travail.

```
rm -f terraform.tfstate terraform.tfstate.backup
```

```
ssh-keygen -t rsa -b 4096 -f ~/.ssh/ma-cle-devops -N ""
```

6. Initialisation du projet

La commande `terraform init` prépare le répertoire de travail. Terraform analyse le code, identifie les fournisseurs nécessaires et télécharge les plugins correspondants. Un provider est le composant qui traduit les instructions Terraform en appels vers l'API du fournisseur cloud ; il en existe pour AWS, Azure, Google Cloud, mais également pour Docker ou GitHub.

```
terraform init
```

```
root@sams-virtual-machine:/home/sams/mon-terraform# terraform init
Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Terraform has been successfully initialized!
```

Figure 6 — Initialisation réussie et installation du provider AWS

7. Prévisualisation des changements

La commande `terraform plan` constitue le principal filet de sécurité de l'outil. Terraform compare le code, l'état enregistré et les ressources réellement présentes sur AWS, puis annonce les actions qu'il compte entreprendre — sans rien modifier.

Le résultat indique ici trois ressources à créer, aucune à modifier et aucune à détruire. La mention « `known after apply` » signale les valeurs qui ne pourront être connues qu'une fois les ressources créées, comme l'adresse IP publique. En entreprise, un plan est systématiquement relu avant toute application en production.

```
terraform plan
```

```
Plan: 3 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + ip_publique = (known after apply)
```

Figure 7 — Plan d'exécution : trois ressources à créer

8. Création de l'infrastructure

La commande `terraform apply` exécute le plan après confirmation. Terraform détermine automatiquement l'ordre de création en fonction des dépendances : la clé SSH et le groupe de sécurité sont créés en premier, l'instance EC2 ensuite, puisqu'elle en dépend. L'ensemble a été créé en moins de vingt secondes.

```
terraform apply
```

```
aws_key_pair.ma_cle: Creating...
aws_security_group.mon_firewall: Creating...
aws_key_pair.ma_cle: Creation complete after 1s [id=ma-cle-devops]
aws_security_group.mon_firewall: Creation complete after 4s [id=sg-026a7fae13f64ce98]
aws_instance.mon_serveur: Creating...
aws_instance.mon_serveur: Still creating... [00m10s elapsed]
aws_instance.mon_serveur: Creation complete after 13s [id=i-0c5a7e509caafa3c6]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:
ip_publique = "51.44.19.240"
```

Figure 8 — Infrastructure créée et adresse IP publique retournée en sortie

9. Vérification dans la console AWS

La console AWS confirme la présence de l'instance mon-serveur-devops, en cours d'exécution, de type t3.micro et associée à l'adresse IP publique retournée par Terraform. Aucune de ces ressources n'a été créée manuellement.

mon-serveur-devops	i-0c5a7e509caafa3c6	En cours d'exécution	t3.micro	3/3 vérificati	eu-west-3c	ec2-51-44-...	51.44.19.240
--------------------	---------------------	----------------------	----------	----------------	------------	---------------	--------------

Figure 9 — Instance EC2 créée automatiquement, visible dans la console AWS

10. Test de l'application

Le script user_data ayant installé Docker et démarré le conteneur au premier démarrage du serveur, l'application est directement accessible depuis un navigateur sur le port 8000, sans aucune intervention manuelle sur la machine.

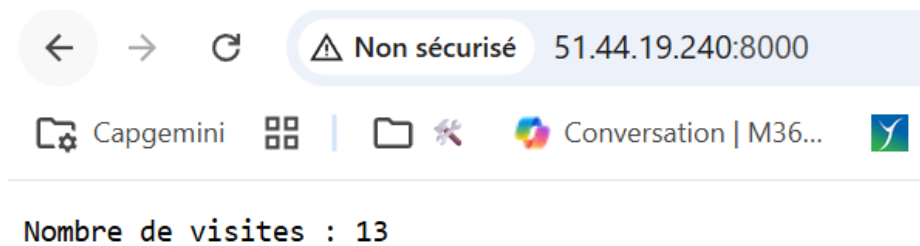


Figure 10 — Application déployée automatiquement et accessible sur Internet

11. Destruction de l'infrastructure

La commande terraform destroy supprime l'ensemble des ressources créées, et uniquement celles-ci : Terraform s'appuie sur son fichier d'état pour savoir précisément ce qu'il a créé. Cette réversibilité est l'un des apports essentiels de l'Infrastructure as Code, tant pour la maîtrise des coûts que pour la reproductibilité des environnements de test.

```
terraform destroy
```

```
Destroy complete! Resources: 3 destroyed.  
root@sams-virtual-machine:/home/sams/mon-terraform#
```

Figure 11 — Destruction complète des trois ressources

12. Points d'attention et bonnes pratiques

- Gestion du fichier d'état : le dépôt contenait un fichier terraform.tfstate correspondant à une infrastructure détruite, qui a dû être supprimé avant relance. Le versionnement d'un fichier d'état constitue une mauvaise pratique reconnue : il peut contenir des informations sensibles en clair et provoque des conflits dès que plusieurs personnes travaillent sur le même projet. En environnement professionnel, il est stocké dans un backend distant, par exemple un bucket S3 associé à un mécanisme de verrouillage.
- Dépendance à l'environnement local : le code référence une clé publique SSH présente sur le poste de travail. Cette dépendance rend le projet non portable en l'état, la clé ayant dû être régénérée sur

le nouveau poste. Une approche plus robuste consisterait à générer la paire de clés depuis Terraform lui-même ou à en externaliser la gestion.

- Restriction des règles réseau : les règles d'entrée autorisent l'ensemble des adresses IP (0.0.0.0/0). Ce paramétrage reste acceptable dans un environnement de test, mais l'accès SSH devrait être limité aux adresses d'administration connues en production, afin de réduire la surface d'exposition.
- Rotation des clés d'accès : les identifiants programmatiques utilisés par Terraform donnent un accès étendu au compte cloud. Une clé exposée doit être désactivée puis supprimée sans délai et remplacée. La rotation régulière de ces identifiants relève de l'hygiène de sécurité en environnement cloud.
- Prévisualisation systématique : l'exécution de terraform plan avant tout apply permet de mesurer l'impact des changements sur l'infrastructure existante. Cette étape est particulièrement critique en production, où une modification mal maîtrisée peut entraîner la destruction et la recréation de ressources en service.

13. Conclusion

Ce projet a permis de mettre en œuvre le cycle complet de l'Infrastructure as Code : écriture du code décrivant l'infrastructure, initialisation, prévisualisation, application, vérification puis destruction. Une seule commande a suffi à créer un serveur cloud, à configurer son pare-feu, à installer Docker et à déployer une application accessible sur Internet.

La comparaison avec une création manuelle depuis la console AWS est éclairante : l'approche par le code est plus rapide, reproductible à l'identique, documentée par nature et intégralement réversible. Ces compétences sont directement transposables à l'administration d'infrastructures en environnement professionnel.